

Italian Restaurant Sales Analytics

03 - Payment Methods Data Cleaning and Preparation

Objective

This notebook cleans and validates the **Payment Methods** worksheet from the Italian Restaurant Sales Analytics dataset. It verifies data quality, checks for missing values and duplicates, validates data types, and exports a cleaned dataset for further analysis in Python, SQL, and Power BI.

Step 1 - Import Libraries

The first step is to import the Python libraries required for data manipulation and analysis.

```
In [1]: # Import pandas for data manipulation and analysis  
import pandas as pd
```

Step 2 - Load the Excel Workbook

The next step is to define the location of the Excel file and load the workbook into Python.

```
In [2]: # Define the relative path to the Excel workbook  
file_path = "../data/raw/restaurant-sales-analytics-data.xlsx"
```

Step 3 - Read the Payment Methods Worksheet

Load the Payment Methods worksheet into a pandas DataFrame.

```
In [3]: # Load the Payment Methods worksheet into a pandas DataFrame  
payment_methods = pd.read_excel(file_path, sheet_name="Payment Methods")
```

Step 4 - Preview the Dataset

Display the first five rows of the dataset.

```
In [4]: # Display the first five rows  
payment_methods.head()
```

```
Out[4]:
```

	Date	Payment Method	Transactions	Sales
0	2025-03-01	TC. DINERS	616	34480.09
1	2025-03-01	TC. VISA	423	24549.31
2	2025-03-01	TD. VISA	382	20143.59
3	2025-03-01	TC. MASTERCARD	329	19137.92
4	2025-03-01	EFFECTIVO	212	7957.73

Step 5 - Inspect the Dataset Structure

Display the structure of the dataset, including data types, number of non-null values, and memory usage.

```
In [5]: # Display dataset information
payment_methods.info()

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 227 entries, 0 to 226
Data columns (total 4 columns):
#   Column          Non-Null Count  Dtype  
---  -
0   Date             227 non-null   datetime64[ns]
1   Payment Method   227 non-null   object  
2   Transactions      227 non-null   int64   
3   Sales            227 non-null   float64  
dtypes: datetime64[ns](1), float64(1), int64(1), object(1)
memory usage: 7.2+ KB
```

Step 6 - Check Dataset Dimensions

Display the number of rows and columns in the dataset.

```
In [6]: # Display dataset dimensions
payment_methods.shape
```

```
Out[6]: (227, 4)
```

Step 7 - Check Missing Values

Identify whether the dataset contains any missing values in each column.

```
In [7]: # Check for missing values
payment_methods.isnull().sum()
```

```
Out[7]: Date          0
        Payment Method 0
        Transactions   0
        Sales          0
        dtype: int64
```

Step 8 - Check for Duplicate Rows

Identify whether the dataset contains duplicate records that may affect the analysis.

```
In [8]: # Check for duplicate rows
        payment_methods.duplicated().sum()
```

```
Out[8]: np.int64(0)
```

Step 9 - Display Column Names

Display all column names in the dataset.

```
In [9]: # Display column names
        payment_methods.columns
```

```
Out[9]: Index(['Date', 'Payment Method', 'Transactions', 'Sales'], dtype='object')
```

Step 10 - Display Summary Statistics

Display descriptive statistics for the numerical columns to better understand the distribution of the data.

```
In [10]: # Display summary statistics
         payment_methods.describe()
```

```
Out[10]:
```

	Date	Transactions	Sales
count	227	227.000000	227.000000
mean	2025-08-30 02:44:56.035242240	125.779736	6837.821806
min	2025-03-01 00:00:00	1.000000	0.000000
25%	2025-06-01 00:00:00	10.000000	380.335000
50%	2025-09-01 00:00:00	46.000000	1344.580000
75%	2025-12-01 00:00:00	206.500000	7585.670000
max	2026-03-01 00:00:00	616.000000	36819.790000
std	NaN	160.888338	10211.244549

Step 11 - Check Unique Payment Methods

Display the unique values in the Payment Methods column to verify that all expected categories are present.

```
In [12]: # Display unique Payment Methods
payment_methods["Payment Method"].unique()
```

```
Out[12]: array(['TC. DINERS', 'TC. VISA', 'TD. VISA', 'TC. MASTERCARD', 'EFFECTIVO',
               'TC.AMEX', 'UBEREATS', 'RAPPI', 'CUENTA X COBRAR', 'PEDIDOS YA',
               'TC. DISCOVER', 'CONSUMO INTERNO', 'TRANSFERENCIA', 'MERMAS',
               'ORDEN DE CONSUMO', 'CANJE', 'BENDO', 'MANUAL', 'FACT REPROCESADA',
               'PAGO CON GIFT CARD', 'CHEQUE', 'CANJE SCALA'], dtype=object)
```

Step 12 - Count Unique Payment Methods

Count the number of unique Payment Methods in the dataset.

```
In [13]: # Count unique Payment Methods
payment_methods["Payment Method"].nunique()
```

```
Out[13]: 22
```

Step 13 - Count Unique Dates

Count the number of unique dates in the dataset.

```
In [14]: # Count unique dates
payment_methods["Date"].nunique()
```

```
Out[14]: 13
```

Step 14 - Count Unique Transactions

Count the number of unique transactions in the dataset.

```
In [16]: # Count unique transactions
payment_methods["Transactions"].nunique()
```

```
Out[16]: 126
```

Step 15 - Display Data Types

Display the data type of each column individually.

```
In [15]: # Display data types
payment_methods.dtypes
```

```
Out[15]: Date                datetime64[ns]
Payment Method              object
Transactions                int64
Sales                      float64
dtype: object
```

Step 16 - Check for Negative Transaction Values

Verify whether the dataset contains any negative transaction values.

```
In [26]: # Check for negative transaction values
(payment_methods["Transactions"] < 0).sum()
```

```
Out[26]: np.int64(0)
```

Step 17 - Check for Zero Transaction Values

Verify whether the dataset contains Transaction Values equal to zero.

```
In [27]: # Check for zero transaction values
(payment_methods["Transactions"] == 0).sum()
```

```
Out[27]: np.int64(0)
```

Step 18 - Verify Date Range

Display the earliest and latest dates in the dataset.

```
In [19]: # Display minimum and maximum dates
payment_methods["Date"].min(), payment_methods["Date"].max()
```

```
Out[19]: (Timestamp('2025-03-01 00:00:00'), Timestamp('2026-03-01 00:00:00'))
```

Step 19 - Sort the Dataset by Date

Sort the dataset in chronological order.

```
In [20]: # Sort the dataset by date
payment_methods = payment_methods.sort_values(by="Date")
```

Step 20 - Reset the Index

Reset the DataFrame index after sorting.

```
In [21]: # Reset the DataFrame index
payment_methods.reset_index(drop=True, inplace=True)
```

Step 21 - Preview the Clean Dataset

Display the first rows of the cleaned dataset.

```
In [22]: # Preview cleaned dataset
payment_methods.head()
```

```
Out[22]:
```

	Date	Payment Method	Transactions	Sales
0	2025-03-01	TC. DINERS	616	34480.09
1	2025-03-01	MANUAL	40	0.00
2	2025-03-01	BENDO	2	0.00
3	2025-03-01	CANJE	1	39.00
4	2025-03-01	ORDEN DE CONSUMO	4	125.00

Step 22 - Export the Clean Dataset

Export the cleaned dataset as a new Excel file for future analysis.

```
In [23]: # Export cleaned Payment Methods dataset
payment_methods.to_excel(
    "../data/cleaned/payment_methods_cleaned.xlsx",
    index=False
)
```

Step 23 - Cleaning Summary

Display a summary of the data quality checks performed during the cleaning process.

```
In [29]: print("=====CLEANING SUMMARY=====")

print(f"Rows: {payment_methods.shape[0]}")
print(f"Columns: {payment_methods.shape[1]}")
print(f"Missing values: {payment_methods.isnull().sum().sum()}")
print(f"Duplicate rows: {payment_methods.duplicated().sum()}")
print(f"Unique Payment Methods: {payment_methods['Payment Method'].nunique()}")
print(f"Negative Transactions: {(payment_methods['Transactions'] < 0).sum()}")
print(f"Zero Transactions: {(payment_methods['Transactions'] == 0).sum()}")
print(f"Date range: {payment_methods['Date'].min().date()} to {payment_methods['Date'].max().date()}")

print("Clean dataset exported successfully.")
print("=====")
```

```
===== CLEANING SUMMARY =====  
Rows: 227  
Columns: 4  
Missing values: 0  
Duplicate rows: 0  
Unique Payment Methods: 22  
Negative Transactions: 0  
Zero Transactions: 0  
Date range: 2025-03-01 to 2026-03-01  
Clean dataset exported successfully.  
=====
```

Author

Guido Acosta

Data Analyst Portfolio Project

GitHub Portfolio – 2026